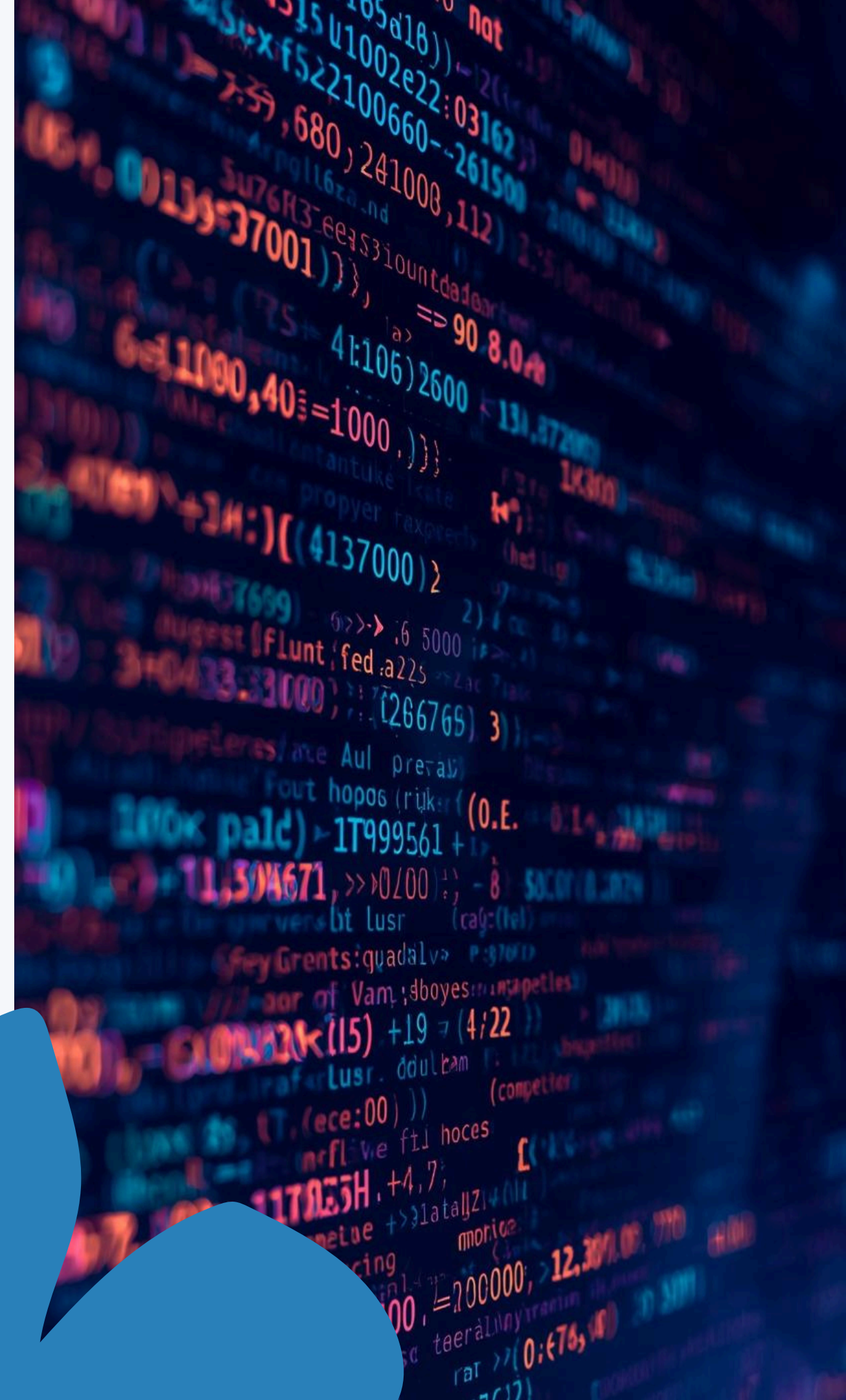


Operaciones Matemáticas con NumPy

JOSÉ ZÚÑIGA



Operaciones aritméticas básicas

NUMPY SOPORTA OPERACIONES ENTRE ARREGLOS DEL MISMO TAMAÑO O ENTRE UN ARREGLO Y UN ESCALAR (BROADCASTING):

```
import numpy as np
x = np.array([[1, 2], [3, 4]])
y = np.array([[5, 6], [7, 8]])

print(x + y)      # Suma
print(x - y)      # Resta
print(x * y)      # Multiplicación elemento a elemento
print(y / x)      # División
```

También se pueden usar funciones equivalentes:

- `np.add(x, y)` → suma
- `np.subtract(x, y)` → resta
- `np.multiply(x, y)` → multiplicación
- `np.divide(x, y)` → división
- Operaciones in-place: `x += y` modifica directamente el arreglo sin crear una nueva copia en memoria

Operaciones vectoriales y matriciales

NumPy implementa operaciones entre vectores y matrices:

Función	Descripción
<code>np.dot(a, b)</code>	Producto punto o matriz
<code>np.inner(a, b)</code>	Producto escalar
<code>np.cross(a, b)</code>	Producto vectorial
<code>np.outer(a, b)</code>	Producto externo
<code>np.kron(a, b)</code>	Producto de Kronecker
<code>@</code>	Operador de multiplicación matricial

```
A = np.array([[1, 2], [3, 4]])  
B = np.array([[5, 6], [7, 8]])  
np.dot(A, B)
```



Operaciones estadísticas

NumPy facilita el cálculo de estadísticas descriptivas:

```
x = np.array([3.5, 1.1, 3.2, 2.8, 6.7, 4.4, 0.9, 2.2])

np.mean(x)      # Promedio
np.median(x)    # Mediana
np.std(x)       # Desviación estándar
np.var(x)       # Varianza
np.min(x)       # Mínimo
np.max(x)       # Máximo
```

Se puede ajustar el parámetro `ddof=1` para obtener la varianza muestral (no poblacional).



Funciones vectorizadas personalizadas

Podemos vectorizar funciones propias con `np.vectorize`:

```
def heaviside(x):  
    return 1 if x > 0 else 0  
  
heaviside = np.vectorize(heaviside)  
x = np.linspace(-5, 5, 11)  
print(heaviside(x))
```

Esto transforma una función escalar en una que opera sobre arreglos completos

Rendimiento: Bucles vs. Vectorización

NumPy permite ejecutar operaciones sobre arreglos completos sin usar bucles explícitos.

Esto se denomina vectorización, y aprovecha optimizaciones internas escritas en C.

Método	Tiempo aproximado
Bucle en Python	~0.25 s
NumPy (np.sum)	~0.002 s

NumPy puede ser hasta 100 veces más rápido al evitar bucles en Python puro.

Broadcasting visual

El broadcasting permite combinar arreglos de distintas dimensiones expandiéndolos automáticamente, sin necesidad de repetir datos.

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([[10], [20], [30]])

print(a + b)
```

```
[[11 12 13]
 [21 22 23]
 [31 32 33]]
```

NumPy extiende a (1×3) a 3×3 para que coincida con b (3×1) .

Esto se hace sin copiar memoria, solo interpretando las dimensiones.